Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- 1. ödevin video anlatımı hakkında bilgilendirme.
- Ağaç (tree) veri yapısının tanımı ve günlük hayattan örnekleri.
- Ağaç terminolojisi: düğüm, kök, ebeveyn, çocuk, yaprak, kardeş, derece, yükseklik, derinlik, seviye, alt ağaç.
- Ağacın dizi ile gösterimi ve bellek israfı.
- Ağacın bağlantılı liste (linked list) ile gösterimi.
- İkili ağaç (binary tree) ve özel türleri: full, complete, perfect, balanced, dejenere.
- İkili ağaca eleman ekleme ve silme.
- Ağaçta gezinti (traversal): preorder, inorder, postorder; yığınla özyinelemesiz inorder; kuyrukla genişlik öncelikli gezinti.
- Ağaç kopyalama ve iki ağacın eşitliğini kontrol etme.
- Çok çocuklu ağaçların (n-ary tree) ilk çocuk–ilk kardeş yöntemiyle ikili ağaca dönüştürülmesi.
- İkili arama ağacı (binary search tree, BST): arama, ekleme, silme.
- Dengeli ağaçlar ve dengeleme yaklaşımları (AVL, red-black, splay, B-tree).

Hocanın Özellikle Vurguladığı Kısımlar

- Ödev videosunda önce öğrenci kendini tanıtmalı, sonra çözüm yöntemini ve alt problemleri nasıl çözdüğünü anlatmalı. “Kod çalışıyor” demek yeterli değil.
- Talimatta 5-10 dakika yazıyor. Hocaya göre iyi hazırlanmış bir anlatım 3 dakikaya sığar.
- Video, değerlendirmede neyin neden yapıldığını görmeyi kolaylaştırıyor. Teknik bir sorun (bulanık ya da boğuk ses) yoksa videoya geri dönüş yapılmaz, doğrudan puanlanır.
- Ağaçta kapalı çevrim (cycle) olamaz. Kökten her düğüme tek bir yol vardır.
- Ağaçlar kök yukarıda, yapraklar aşağıda olacak şekilde ters çizilir.
- Kardeş düğümlerin derinliği aynıdır ama yükseklikleri alt soylarına göre farklı olabilir. Yükseklik ve derinlik sık karıştırılır.
- Dizi ile ağaç gösterimi, ağaç tam dolu değilse çok fazla bellek israf eder. Bu yüzden linked list gösterimi tercih edilir.
- Her ağaç ilk çocuk–ilk kardeş yöntemiyle ikili ağaç olarak gösterilebilir. Böylece ikili ağaç için yazılan fonksiyonlar bütün ağaçlarda kullanılabilir.
- BST’de inorder gezinti elemanları sıralı verir.
- BST’den iki çocuklu düğüm silinirken ya sağ alt ağacın en küçüğü ya da sol alt ağacın en büyüğü yukarı taşınır.
- Dengesiz BST’de işlemler $O(n)$’e kadar çıkar. Dengeleme algoritmaları yüksekliği $O(\log n)$ civarında tutar. Dengeleme algoritmaları bu dersin sorumluluğunda değil.
- Hoca ağaç oluşturma, ekleme, silme ve gezinti kodlarının öğrenciler tarafından mutlaka yazılmasını istedi.

Kısa Tekrar Notları

- **Ağaç:** Hiyerarşik yapıda birbirine bağlı, kapalı çevrim içermeyen sonlu düğüm kümesi. Grafın özel bir hâlidir.
- **Kök (root):** Atası olmayan tek düğüm.
- **Ebeveyn (parent):** Çocuğu olan düğüm. **Çocuk (child):** Bir ebeveyne bağlı düğüm.

- **Yaprak (leaf, external node):** Çocuğu olmayan düğüm. **İç düğüm (internal node):** Çocuğu olan düğüm.
- **Kardeş (sibling):** Aynı ebeveynin çocukları.
- **Derece (degree):** Bir düğümün çocuk sayısı. Ağacın derecesi, düğüm derecelerinin en büyüğüdür.
- **Derinlik (depth):** Düğümden köke kadar olan kenar sayısı. Kökün derinliği 0'dır.
- **Yükseklik (height):** Düğümden en uzak yaprağa kadar olan kenar sayısı. Yaprağın yüksekliği 0, ağacın yüksekliği kökün yüksekliğidir.
- **Seviye (level):** Derste seviye = derinlik + 1 olarak verildi; kök 1. seviyededir.
- **Alt ağaç (subtree):** Bir düğüm ve bütün alt soyu.
- **Serbest ağaç (free tree):** Kökü belirlenmemiş ağaç. Bir düğüm kök seçilince hiyerarşi oluşur.
- **İkili ağaç türleri:**
  - Full: her düğümün 0 ya da 2 çocuğu var.
  - Complete: son seviye hariç bütün seviyeler dolu, son seviye soldan dolu.
  - Perfect: bütün iç düğümlerin 2 çocuğu var ve bütün yapraklar aynı seviyede.
  - Balanced: yükseklik $O(\log n)$ mertebesinde.
  - Dejenere / patolojik: her düğümün tek çocuğu var; ağaç linked list'e dönüşür.
- **0 tabanlı dizide ikili ağaç:** `j` indisli düğümün sol çocuğu `2j+1`, sağ çocuğu `2j+2`, ebeveyni $\lfloor (j-1)/2 \rfloor$.
- **Gezintiler:**
  - Preorder: visit, left, right (VLR).
  - Inorder: left, visit, right (LVR).
  - Postorder: left, right, visit (LRV).
  - Ayna sıralar (VRL, RVL, RLV) aynı gezintilerin sağdan sola hâlidir.
- **Genişlik öncelikli (breadth first, level order):** Kuyruk ile seviye seviye, soldan sağa gezilir.
- **Ağaç kopyalama:** postorder. **Ağaç eşitliği:** preorder.
- **BST:** Her düğümde sol alt ağaçtaki bütün değerler küçük, sağ alt ağaçtaki bütün değerler büyüktür. Alt ağaçlar da BST'dir.
- **BST arama, ekleme, silme:** $O(h)$. Dengeli ağaçta $O(\log n)$, dejenere ağaçta $O(n)$.

Detaylı Açıklamalar

Ağacın Tanımı

İki tanım verildi:

1. Hiyerarşik yapıda birbirine bağlı ve kapalı çevrim oluşturmayan sonlu düğüm kümesi.
2. Aşağıdaki özellikleri taşıyan düğümler topluluğu:
   - kendine doğru kenar içermeyen tek bir kök düğüm vardır,
   - kök dışındaki her düğüme tek bir kenar gelir,
   - kökten her düğüme tek bir yol vardır.

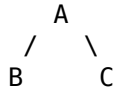
Bir düğüme iki farklı yoldan gidilebiliyorsa çevrim vardır ve yapı ağaç değildir.

Günlük hayattan örnekler:

- soy ağacı,
- şirket organizasyon şeması,
- tenis turnuvası eşleşmeleri (128 kişide 4 tur, ardından çeyrek final, yarı final, final),
- kitabın içindekiler tablosu,
- dosya sistemi,
- veri tabanı indeksleri.

Terminoloji Örneği

Derste kullanılan ağaç (A kök):



```
 / \  / | \
D  E F  G  H
           |
           I
```

- A kök ve ebeveyndir.
- B, D ve E'nin ebeveynidir.
- C, A'nın çocuğu ve B'nin kardeşidir.
- D ve E, B'nin çocukları ve A'nın torunlarıdır.
- D, E, F, G ve I yapraktır.
- A, B, C ve H iç düğümdür.
- E'nin seviyesi 3'tür.
- Ağacın yüksekliği 3'tür (A → C → H → I).
- B'nin derecesi 2'dir. Ağacın derecesi 3'tür (C'nin üç çocuğu var).
- I'nın ataları H, C ve A'dır.
- C'nin alt soyu F, G, H ve I'dır.

Dizi ile Ağaç Gösterimi

Yöntem 1, sol ve sağ indis tutmak: Her gözde veri, sol çocuğun indisi ve sağ çocuğun indisi tutulur. Çocuk yoksa `-1` yazılır. Ayrıca kökün indisi ve bir sonraki boş gözün indisi tutulur.

İndis	Veri	Sol	Sağ
0	Jane	1	2
1	Bob	3	4
2	Tom	5	-1
3	Alan	-1	-1
4	Ellen	-1	-1
5	Nancy	-1	-1

Kök 0, boş göz 6'dır. Slaytta boş gözler de sağ indis alanıyla birbirine bağlanmış: 6 → 7 → 8 → 9 → ...

- **Mary'yi Nancy'nin sol çocuğu olarak eklemek:** Mary 6. göze yazılır, Nancy'nin sol indisi 6 yapılır, boş göz 7 olur.
- **4. gözdeki düğümü silmek:** Bob'un sağ indisi `-1` yapılır ve 4. göz boş olarak işaretlenir.

```
struct node {
    int data;
    int left;
    int right;
};
struct node tree[100];
int root, bos;
```

Yöntem 2, konuma göre yerleştirmek: 0 tabanlı dizide `j` indisli düğümün sol çocuğu `2j+1`, sağ çocuğu `2j+2` indisindedir. Ebeveyni $\lfloor (j-1)/2 \rfloor$ indisindedir.

Bu yöntemde ağaç tam dolu değilse çok yer boşa gider:

- Yüksekliği h olan ikili ağaç için $2^{h+1} - 1$ göz açılmalıdır.
- Derste 4 elemanlı bir ağaç için 15 göz açılan, 11 gözün boş kaldığı örnek gösterildi.
- En kötü durumda her düğümün tek çocuğu vardır. O zaman n eleman için $2^n - 1$ göz gerekir.

Linked List ile İkili Ağaç

Her düğümde veri ile sol ve sağ çocuk işaretçisi bulunur. n eleman için n düğüm yeterlidir.

```
typedef struct node *tree;
struct node {
```

```
    char data;
    tree left;
    tree right;
};

tree createTree(char data) {
    tree t = (tree)malloc(sizeof(struct node));
    t->data = data;
    t->left = NULL;
    t->right = NULL;
    return t;
}
```

İkili ağacın özyinelemeli tanımı: Boş küme bir ikili ağaçtır. Ya da bir kök ile bu köke bağlı sol ve sağ ikili alt ağaçlardan oluşan sonlu düğüm kümesidir.

İkili Ağaç Türleri

- **Full binary tree:** Tek çocuklu düğüm yoktur. Derste 18 kökü olan ağaç örnekti.
- **Complete binary tree:** Son seviye hariç bütün seviyeler doludur. Son seviye soldan doldurulmuştur. Tek çocuklu bir düğüm olsa da complete olabilir.
- **Perfect binary tree:** Hem full hem complete'tir. Bütün yapraklar aynı seviyededir.
- **Balanced binary tree:** Yükseklik $O(\log n)$'dir.
- **Dejenere (patolojik) ağaç:** Her düğümün tek çocuğu vardır. Ağaç linked list gibi davranır.

Örnek: A'nın çocukları B ve C; B'nin çocukları D ve E; C'nin çocukları F ve G; D'nin çocukları H ve I. Bu ağaç 9 düğümlüdür.

- Full'dür: her düğümün 0 ya da 2 çocuğu var.
- Complete'tir: son seviye soldan dolu.
- Perfect değildir: son seviye tam dolu değil.
- Dengelidir: yükseklik 3, $\log_2 9 \approx 3{,}17$.

İkili Ağaca Ekleme ve Silme

Ekleme için bir sıralama kuralı gerekir: küçükler sola, büyükler sağa. Böylece ikili arama ağacı oluşur.

```
tree insert(tree node, char data) {
    if (node == NULL)
        return createTree(data);
    if (data < node->data)
        node->left = insert(node->left, data);
    else if (data > node->data)
        node->right = insert(node->right, data);
    return node;   /* eşitse hiçbir şey yapılmaz */
}
```

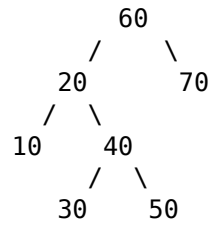
Örnek: 6, 4, 5, 8 sırasıyla eklenir.

- 4, 6'nın soluna girer.
- 5, 4'ün sağına girer.
- 8, 6'nın sağına girer.
- 4 tekrar gelirse eşit olduğu için eklenmez.

Silme üç duruma ayrılır: yaprak, tek çocuklu düğüm, iki çocuklu düğüm. Ayrıntılar BST bölümündedir.

Derinlik Öncelikli Gezintiler (Depth First Traversal)

Örnek ağaç (aynı zamanda bir BST):



Gezinti	Sıra	Sonuç
Preorder	V L R	60, 20, 10, 40, 30, 50, 70
Inorder	L V R	10, 20, 30, 40, 50, 60, 70
Postorder	L R V	10, 30, 50, 40, 20, 70, 60

```c
void preTraverse(tree t) {
    if (t == NULL) return;
    printf("%c", t->data);   /* visit */
    preTraverse(t->left);
    preTraverse(t->right);
}

void inTraverse(tree t) {
    if (t == NULL) return;
    inTraverse(t->left);
    printf("%c", t->data);
    inTraverse(t->right);
}

void postTraverse(tree t) {
    if (t == NULL) return;
    postTraverse(t->left);
    postTraverse(t->right);
    printf("%c", t->data);
}
```

Özyinelemesiz inorder: Yığın kullanılarak function call stack elle taklit edilir. Fikir şudur:

1. Gidilebildiği kadar sola gidilir ve geçilen düğümler yığına itilir.
2. Sol bitince yığından bir düğüm çekilir ve ziyaret edilir.
3. Sonra o düğümün sağ çocuğuna geçilir.

Genişlik öncelikli gezinti (breadth first): Kuyruk kullanılır.

1. Kök kuyruğa eklenir.
2. Kuyruktan bir düğüm çıkarılır ve kullanılır.
3. Düğümün çocukları soldan sağa kuyruğa eklenir.
4. Kuyruk boşalana kadar devam edilir.

Derste kökü F olan ağaçta sıra F, C, J, A, D, H, K, I oldu.

Ağaç Kopyalama ve Eşitlik

- **Kopyalama:** Önce yapı (düğümler) oluşturulmalı, sonra veri kopyalanmalıdır. Bu yüzden postorder kullanılır.
- **Eşitlik kontrolü:** Önce kökteki veriler karşılaştırılır. Eşit değilse alt ağaçlara inmeye gerek kalmaz. Bu yüzden preorder daha uygundur.

```c
int isEqualTree(tree first, tree second) {
    return ((!first && !second) ||
            (first && second &&
             first->data == second->data &&
             isEqualTree(first->left, second->left) &&
```

```
            isEqualTree(first->right, second->right)));
}
```

Çok Çocuklu Ağaçların İkili Ağaç ile Gösterimi

Her düğümde yalnız iki işaretçi tutulur:

- ilk çocuk (first child),
- bir sonraki kardeş (next sibling).

Örnek:

- A'nın ilk çocuğu B'dir. B'nin kardeşleri sırasıyla C, D, E, F, G'dir.
- B'nin ilk çocuğu H'dir. H'nin kardeşi I'dır.

Böylece derecesi ne olursa olsun her ağaç `data`, `child`, `sibling` alanlı düğümlerle ikili ağaç olarak saklanır.

İkinci örnek, dörtlü bir ağaçtı:

- V0'ın ilk çocuğu V1'dir. V1'in kardeşleri V2, V3, V4'tür.
- V1'in ilk çocuğu V5, V5'in kardeşi V6'dır.
- V3'ün ilk çocuğu V7, V7'nin kardeşleri V8 ve V9'dur.
- V7'nin ilk çocuğu V10'dur.

İkili Arama Ağacı (BST)

Tanım: Her düğümün değeri, sol alt ağacındaki bütün değerlerden büyük, sağ alt ağacındaki bütün değerlerden küçüktür. Sol ve sağ alt ağaçlar da BST'dir.

Kontrol örnekleri:

- Kökü 20 olan ağaçta 10, 15'in sağında durmalıyken solunda duruyordu. Bu yüzden BST değildir.
- 30, 40, 5, 2 değerlerinden oluşan ağaç dengesizdir ama BST'dir.
- 60, 70, 80, 65 değerlerinden oluşan ağaç BST'dir.

Arama:

```
tree search(tree root, int data) {
    if (root == NULL || root->data == data)
        return root;
    if (data < root->data)
        return search(root->left, data);
    return search(root->right, data);
}
```

Derste kökü 44 olan ağaçta iki arama yapıldı:

- 25 için yol 44 → 17 → 32 → 28 → `NULL` oldu. Değer bulunamadı.
- 76 için yol 44 → 88 → 65 → 82 → 76 oldu. Değer bulundu.

Silme (kökü 94 olan örnek ağaçta):

1. **Yaprak (5):** Doğrudan silinir ve `free` edilir. Ebeveynin işaretçisi `NULL` yapılır.
2. **Tek çocuklu düğüm (24 ya da 11):** Çocuk, silinen düğümün ebeveynine bağlanır ve düğüm silinir.
3. **İki çocuklu düğüm (10):** İki yol vardır.
   - Sağ alt ağacın en küçüğü bulunur: sağa bir kez, sonra gidilebildiği kadar sola gidilir. Burada 11'dir. 11, 10'un yerine kopyalanır. Sonra 11 sağ alt ağaçtan silinir; 11'in sağ çocuğu 17 yukarı bağlanır.
   - Ya da sol alt ağacın en büyüğü (burada 5) kopyalanıp silinir.

Slayttaki silme kodu:

```
tree delete(tree T, char data) {
    if (T == NULL)
        return NULL;
```

```
    if (data < T->data)
        T->left = delete(T->left, data);
    else if (data > T->data)
        T->right = delete(T->right, data);
    else if (T->left != NULL && T->right != NULL) {
        T->data = findMin(T->right)->data;
        T->right = delete(T->right, T->data);
    }
    else
        T = (T->left != NULL) ? T->left : T->right;
    return T;
}
```

`findMin`, `NULL` görene kadar sola gider. `findMax`, `NULL` görene kadar sağa gider.

Dengeleme

Rastgele ekleme ve silmelerle BST'nin yüksekliği ortalamada yaklaşık $\log n$ olur. Ancak en kötü durumda n'e çıkar. Arama, ekleme ve silme $O(h)$ sürdüğü için bu $O(n)$ demektir.

Dengeleme algoritmaları yüksekliği $O(\log n)$ civarında tutar:

- AVL,
- red-black tree,
- splay tree,
- B-tree, 2-3 ve 2-3-4 ağaçları.

Örnek: 4, 6, 8 sırayla eklenirse ağaç sağa doğru bir zincire döner. AVL bu durumda saat yönünün tersine bir dönüş (rotation) yapar. 6 kök, 4 sol çocuk, 8 sağ çocuk olur.

B-tree türleri veri tabanı indekslerinde standart olarak kullanılır.

Ders Sonrası Notlar

- **Özyinelemesiz inorder slaytı:** Slayttaki 2. ve 3. adımlar birer kez çalışacak gibi yazılmış. Bir düğümün sağ çocuğu varsa 3. döngüden çıkılır ve algoritma yarıda kalır. Doğrusu iki adımı bir dış döngüye almaktır:

  ```
  current = root;
  while (current != NULL || !isEmpty()) {
      while (current != NULL) {
          push(current);
          current = current->left;
      }
      x = pop();
      printf("%c", x->data);
      current = x->right;
  }
  ```

- **"Her seviyede inorder":** Slaytta genişlik öncelikli gezinti "her seviyede inorder gezinti" diye tanımlanmış. Doğru terim seviye sırası (level order) gezintidir. Her seviye soldan sağa gezilir; inorder ile ilgisi yoktur.
- **Eşitlik kodunun ilk şartı:** Hoca `(!first && !second)` için "biri null ise eşitlik olamaz" dedi. Bu şart, iki ağacın birlikte boş olduğu durumdur ve `true` döndürür. Yalnız biri boşsa ikinci parantez `false` olur.
- **Yükseklik tanımı:** Hoca bir yerde tek düğümlü ağacın yüksekliğini 1 aldı, başka yerde yaprağın yüksekliğini 0 verdi. Kenar sayısıyla tanımda yaprak ve tek düğümlü ağacın yüksekliği 0'dır. Düğüm sayısıyla tanımda 1'dir. Hangi tanımın kullanıldığı her zaman belirtilmelidir.
- **Ağacın derecesi:** Hoca köklü ağaç örneğinde bir yerde "bu ağacın seviyesi 4, yani 4'lü ağaç" dedi. Burada kastedilen derecedir (bir düğümün en fazla 4 çocuğu var).

- $\log_2 9$**:** Hoca bu değeri "4 nokta bir şey" dedi. Doğrusu $\log_2 9 \approx 3{,}17$'dir. Örnek ağacın yüksekliği 3'tür ve dengeli sonucu değişmez.
- **Dizi gösteriminde göz sayısı:** "n eleman için $2^n - 1$ göz" ifadesi yalnız en kötü (dejenere) durum için geçerlidir. Genel ifade, yüksekliği h olan ikili ağaç için $2^{h+1} - 1$ gözdür. Üçlü ağaçta $3^n - 1$ değil, $(3^{h+1} - 1)/2$ göz gerekir.
- **Dizide silinen göz:** Derste silinen 4. göz doğrudan "bir sonraki boş göz" yapıldı. Bu durumda önceki boş göz bilgisi (7) kaybolur. Slayttaki gibi boş gözler sağ indisle zincirlenmişse silinen gözün sağ indisine eski boş göz (7) yazılmalı, sonra `bos = 4` yapılmalıdır. Bu bir boş göz listesidir (free list).
- **Silme kodunda bellek ve yazım:** Slayttaki kodda tek çocuklu ya da yaprak durumda `T` yeni değerle değiştiriliyor ama eski düğüm `free` edilmiyor. Ayrıca `t->right` satırı `T->right` olmalıdır.
- **Genişlik öncelikli örnek:** Derste sıra "F, C, C, A, ..." diye okundu. Doğru sıra F, C, J, A, D, H, K, I'dır.

Duyurular: - Ders yüz yüze sınıfta başlayacaktı. Sınıfta kimse olmadığı için ders online devam etti. - 1. ödevin teslimi tamamlandı. Sonraki ödevlerde de anlatım videosu istenecek.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.